

Smart Hotel Booking

التوثيق الفني الشامل لهيكلية المشروع والنظام

تم التوليد بناءً على الفحص الفعلي للملفات

٢٠٢٦/٨/٢٢



فهرس المحتويات

- 1. نظرة عامة على المشروع
- 2. التقنيات المستخدمة (Technology Stack)
- 3. الهيكلية العامة للنظام (System Architecture)
- 4. المخطط الشجري للمشروع (Project Tree)
- 5. هيكلية وتفاصيل الواجهة الأمامية (Frontend)
- 6. هيكلية وتفاصيل الواجهة الخلفية (Backend)
- 7. مسارات النظام (API Endpoints)
- 8. بنية قاعدة البيانات والنماذج (Database Architecture)
- 9. المصادقة والصلاحيات (Auth & Authorization)
- 10. الأنظمة الوظيفية الرئيسية (نظام الحجز والتسعر والولاء)
- 11. الخلاصة

1. نظرة عامة على المشروع

نظام Smart Hotel Booking هو منصة متكاملة لإدارة حجوزات الفنادق، يوفر تجربة مستخدم سلسلة للعملاء للبحث عن الفنادق، مقارنتها، إجراء الحجوزات، وكسب نقاط الولاء. كما يوفر النظام لوحات تحكم متقدمة لمديري الفنادق لإدارة الغرف والأسعار والتسعر الديناميكي، ولوحة تحكم شاملة لمسؤولي النظام لإدارة المستخدمين والصلاحيات. تم بناء هذا التوثيق عبر فحص الملفات الفعلية في الخادم وتتبع مسار البيانات الفعلي دون أي افتراضات خارجية.

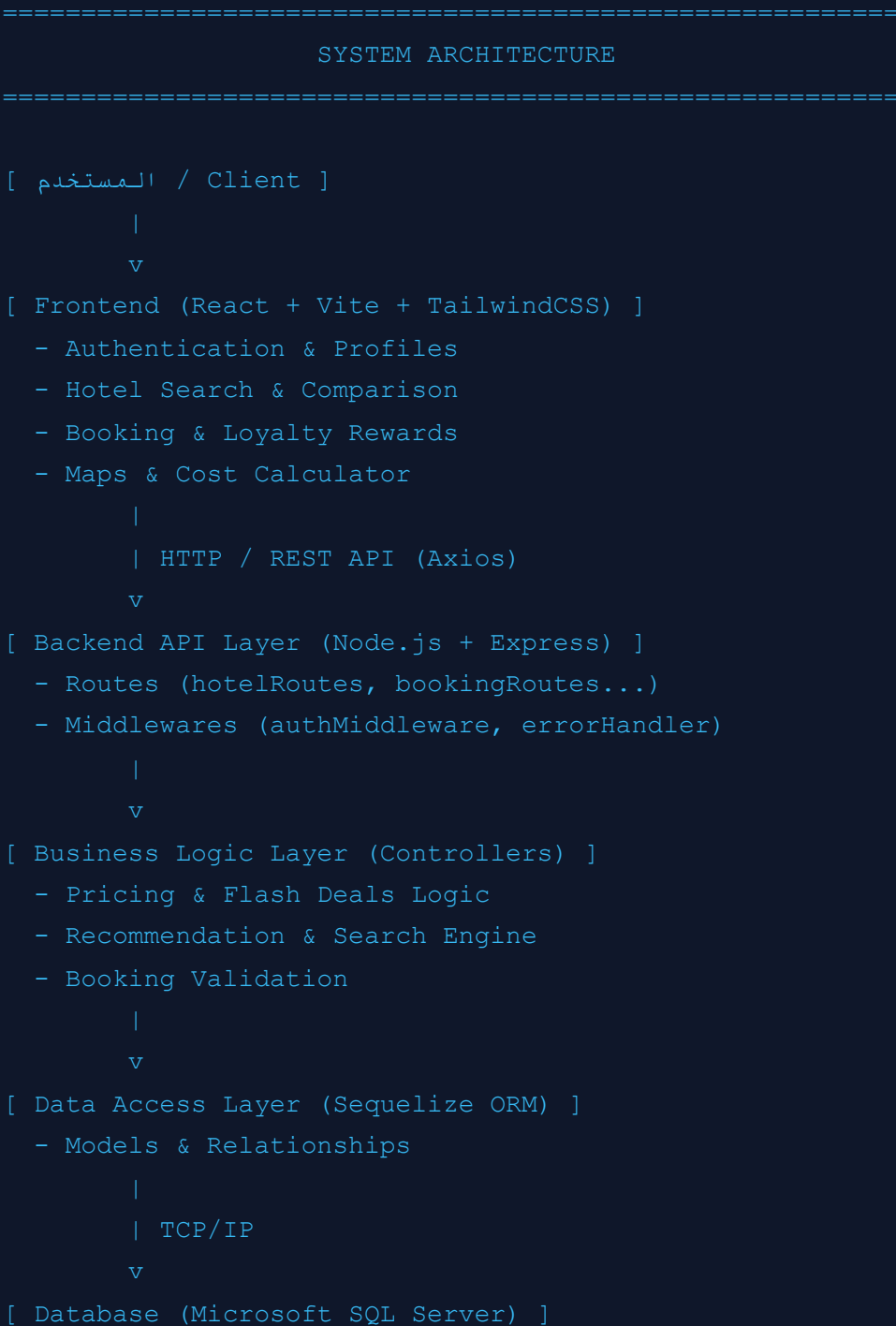
2. التقنيات المستخدمة (Technology Stack)

الطبقة	التقنية الفعلية المستخدمة
الواجهة الأمامية (Frontend)	React.js, Vite, Tailwind CSS, React Router, Context API, Lucide React (Icons), React Leaflet (Maps)
الواجهة الخلفية (Backend)	Node.js, Express.js
قاعدة البيانات (Database)	Microsoft SQL Server
التعامل مع البيانات (ORM)	Sequelize ORM

JSON Web Tokens (JWT), bcryptjs	المصادقة (Authentication)
Multer	رفع الملفات (File Upload)
Docker, Docker Compose	التشغيل والحاويات (DevOps)

3. الهيكلية العامة للنظام (System Architecture)

يعتمد المشروع على بنية متدرجة (Layered Architecture) تفصل واجهة المستخدم عن منطق الأعمال وقاعدة البيانات، معتمدة على حاويات Docker لتنظيم التشغيل.



4. المخطط الشجري الفعلي للمشروع (Project Tree)

```
SMART HOTEL PRO
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   ├── context/
│   │   ├── hooks/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── App.css
│   │   ├── App.jsx
│   │   ├── index.css
│   │   └── main.jsx
│   ├── .oxlintrc.json
│   ├── Dockerfile
│   ├── index.html
│   ├── package.json
│   ├── postcss.config.js
│   ├── tailwind.config.js
│   └── vite.config.js
├── backend/
│   ├── src/
│   │   ├── config/
│   │   ├── controllers/
│   │   ├── middlewares/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── seed/
│   │   ├── utils/
│   │   ├── validators/
│   │   ├── app.js
│   │   └── server.js
│   ├── .dockerignore
│   ├── .env
│   ├── clear_bookings.js
│   ├── Dockerfile
│   └── package.json
├── .env
├── .gitignore
├── alter.sql
├── docker-compose.yml
├── err.log
└── log.txt
```

└─ testLoyalty.js

5. هيكلية وتفاصيل الواجهة الأمامية (Frontend)

الوظيفة والمسؤولية	الملف (File)
يعرض الفنادق المختارة للمقارنة في درج جانبي ويسمح بإدارتها.	src/components/ComparisonDrawer.jsx
مكوّن قابل لإعادة الاستخدام لعرض بطاقة الفندق والصورة والتقييم والسعر.	src/components/HotelCard.jsx
شريط التنقل الرئيسي وروابط الأقسام وحالة تسجيل الدخول.	src/components/Navbar.jsx
يحمي المكونات والصفحات حسب صلاحية المستخدم (عميل، مدير، مسؤول).	src/components/RoleGuard.jsx
يدير حالة المصادقة الشاملة ويحفظ بيانات المستخدم والتوكن في التخزين المحلي.	src/context/AuthContext.jsx
يدير قائمة الفنادق المختارة للمقارنة لتبقى متاحة بين الصفحات.	src/context/ComparisonContext.jsx
خطاف مخصص لتحويل وعرض أسعار الفنادق وعملياتها بشكل متناسق.	src/hooks/useCurrency.js
لوحة تحكم مسؤول النظام لإدارة وتعديل بيانات جميع المستخدمين وتعيين المديرين.	src/pages/AdminPortal.jsx
صفحة تعرض مقارنة مفصلة للمواصفات والأسعار بين عدة فنادق.	src/pages/Compare.jsx
تعرض الفنادق التي وضعها المستخدم في قائمة مفضلاته للرجوع إليها لاحقاً.	src/pages/Favorites.jsx
تعرض تفاصيل فندق معين وغرفته وموقعه على الخريطة الحقيقية وتسمح بإتمام الحجز.	src/pages/HotelDetail.jsx
صفحة البحث الرئيسية مع محرك تصفية متقدم حسب السعر والمدينة والمرافق.	src/pages/Hotels.jsx
واجهة تسجيل الدخول والتحقق من حساب المستخدم الحالي.	src/pages/Login.jsx

تعرض تفاصيل نقاط الولاء ومستوى المستخدم والمكافآت المتوفرة لاستبدالها.	src/pages/Loyalty.jsx
لوحة تحكم مخصصة لمدير الفندق لتعديل الأسعار وإدارة الغرف وتحديد موقع الخريطة.	src/pages/ManagerPortal.jsx
تستعرض حجوزات المستخدم النشطة والمنتوية.	src/pages/MyBookings.jsx
صفحة لعرض وتحديث بيانات الحساب الشخصي للمستخدم.	src/pages/Profile.jsx
واجهة إنشاء حساب جديد.	src/pages/Register.jsx
أداة مساعدة لتقدير الميزانية الإجمالية للرحلة بناء على تكاليف المدينة المحددة.	src/pages/TripCostCalculator.jsx
إعدادات مكتبة Axios وتصدير وظائف الاتصال بواجهات Backend API المركزية.	src/services/api.js
المكون الجذري الذي يعرف مسارات تطبيق React وربط الصفحات بالروابط.	src/App.jsx
نقطة إقلاع التطبيق حيث يتم تغليفه بمزودات السياق (Context Providers).	src/main.jsx

6. هيكلية وتفاصيل الواجهة الخلفية (Backend)

المتحكمات (Controllers)

الوظيفة والمسؤولية	الملف
إدارة عمليات مسؤول النظام الشاملة كالتحكم بالمستخدمين وتخصيص الفنادق للمديرين.	adminController.js
إدارة وجلب قوائم المرافق والخدمات المتاحة للفنادق.	amenityController.js
معالجة تسجيل الدخول وإنشاء الحسابات الجديدة وإصدار رموز JWT.	authController.js
مسؤول عن إنشاء الحجوزات وحساب السعر النهائي وتطبيق التسعير الديناميكي والعروض ونقاط الولاء والضرية.	bookingController.js
جلب قائمة المدن والوجهات السياحية المتاحة وتكاليفها.	cityController.js
يتيح للمستخدمين حفظ واسترجاع مجموعات المقارنة التي أنشأوها سابقاً.	comparisonController.js
مسؤول عن إضافة وإزالة واسترداد الفنادق من قائمة مفضلة المستخدم.	favoriteController.js
تنفيذ عمليات البحث المعقدة وتصفية وجلب الفنادق وتفاصيلها الكاملة للعملاء.	hotelController.js
إدارة رصيد نقاط الولاء للمستخدم ومستواه وعرض واسترداد المكافآت المستحقة.	loyaltyController.js
إدارة عمليات مدير الفندق كتحديث معلومات الفندق وإضافة الغرف وتعديل الإحداثيات وتطبيق التسعير الديناميكي.	managerController.js
توليد قائمة بالفنادق الموصى بها للمستخدم بناءً على الاهتمامات.	recommendationController.js
استقبال وحفظ وجلب تقييمات ومراجعات الزوار للفنادق.	reviewController.js

الإعدادات والوظائف الإضافية (Config, Middlewares, Utils)

الوظيفة والمسؤولية	الملف
إعداد الاتصال بقاعدة بيانات Microsoft SQL Server باستخدام Sequelize.	config/database.js
التحقق من توكن المستخدم وصلاحياته قبل السماح بالمرور للعمليات المحمية.	middlewares/authMiddleware.js
اصطياد الأخطاء وتوحيد ردود الخادم (Responses) عند حدوث مشكلة.	middlewares/errorHandler.js
توليد رموز مصادقة صالحة وأمنة لتسجيل الدخول.	utils/jwtHelper.js
التحقق القبلي من صحة وشكل البيانات المرسلة عند التسجيل وتصحيحها.	validators/authValidators.js
سكربت يحقن قاعدة البيانات الفارغة بالبيانات الافتراضية والمدن لتهيئة النظام.	seed/autoSeed.js
إنشاء الخادم وربط المسارات (Routes) وفتح الاتصال بالمنفذ المخصص للـ Backend.	server.js & app.js

7. مسارات النظام (API Endpoints)

تم استخراج هذه المسارات الفعلية من ملفات الـ routes.

الطلب (Method)	المسار (Endpoint)	المتحكم (Controller)
POST	api/auth/register/	authController.register
POST	api/auth/login/	authController.login
GET	api/auth/me/	authController.getMe
GET	api/hotels/	hotelController.getAllHotels
GET	api/hotels/:id/	hotelController.getHotelById
GET	api/hotels/:id/rooms/	hotelController.getHotelRooms
POST	api/bookings/	bookingController.createBooking
GET	api/bookings/my-bookings/	bookingController.getMyBookings
GET	api/loyalty/my-loyalty/	loyaltyController.getMyLoyalty
POST	api/loyalty/redeem-reward/	loyaltyController.redeemReward
GET	api/manager/my-hotel/	managerController.getMyHotel
PUT	api/manager/my-hotel/	managerController.updateMyHotel
POST	api/manager/rooms/	managerController.createRoom
POST	api/manager/pricing-rules/	managerController.createPricingRule
POST	api/favorites/	favoriteController.addFavorite
GET	api/favorites/	favoriteController.getFavorites
DELETE	api/favorites/:id/	favoriteController.removeFavorite

comparisonController.saveComparison	api/comparisons/saved/	POST
comparisonController.getSaved	api/comparisons/saved/	GET
adminController.getAllUsers	api/admin/users/	GET
adminController.assignManager	api/admin/assign-manager/	PUT

8. بنية قاعدة البيانات والنماذج (Database Architecture)

يستخدم المشروع Sequelize ORM لإدارة جداول Microsoft SQL Server بطريقة ارتباطية ممتازة.

النموذج الفعلي (Model)	وظيفته في النظام
User	يمثل بيانات المستخدمين الأساسية، وكلمات المرور المشفرة، ويحدد الدور (admin, manager, customer).
Hotel	يحتوي على اسم الفندق، الوصف، الإحداثيات الجغرافية الدقيقة، ومفتاح المدينة.
Room	يرتبط بالفندق، ويحتوي على نوع الغرفة، السعر الأساسي، والسعة القصوى.
Booking	يربط المستخدم بالغرفة والفندق، ويسجل تواريخ الدخول والخروج، والسعر الإجمالي الذي تم حسابه.
City	يخزن بيانات المدن وتكاليف المعيشة اليومية اللازمة لتشغيل (Trip Cost Calculator).
DynamicPricingRule	يحفظ قواعد رفع أو خفض السعر (Multiplier) في المواسم المحددة والتي يضعها مدير الفندق.
Favorite	جدول ربط (Many-to-Many) بسيط لحفظ فنادق الإعجاب للمستخدمين.
LoyaltyReward UserRewardInstance	يحددان المكافآت المتاحة في النظام، والمكافآت التي قام المستخدم باستردادها لاستخدامها في حجز قادم.
SavedComparison	يحفظ مصفوفة أرقام الفنادق (hotel_ids) تحت اسم مخصص، ليتيح للعميل العودة لمقارنته لاحقاً.

9. المصادقة والصلاحيات (Auth & Authorization)

يتم التعامل مع الحماية عبر:

- التشفير:** تشفير كلمات المرور باستخدام `bcryptjs` في `User.js` قبل حفظها.
- الرموز:** إصدار رموز `JWT` صالحة ليوم واحد عبر `jwtHelper.js` عند نجاح المصادقة.
- الأدوار الفعلية في الكود:** `admin` (مسؤول النظام)، `manager` (مدير فندق)، `customer` (عميل).
- حماية الواجهة الخلفية:** يستخدم `authMiddleware.js` للتحقق من التوكن، و `authorizeRoles` لمنع العميل من الدخول لمسارات المدير.
- حماية الواجهة الأمامية:** يعتمد مكون `RoleGuard.jsx` على دور المستخدم المخبأ لإخفاء الصفحات الإدارية وطرده المتطفلين وتوجيههم لصفحة الخطأ أو الرئيسية.

10. الأنظمة الوظيفية الرئيسية في المشروع

أ. تدفق حساب الحجز والتسعير (Booking & Pricing Flow)

بناءً على الكود في `bookingController.js`، يتم حساب السعر لكل ليلة إقامة بدقة بالترتيب التالي:

- السعر الأساسي:** جلب سعر الغرفة `room.price_per_night`.
- التسعير الديناميكي:** التحقق من وجود قاعدة في `DynamicPricingRule` تتوافق مع التواريخ (مثل أيام الذروة أو عطلة الأسبوع) وتطبيق المضاعف (`Multiplier`) لتعديل السعر الأساسي صعوداً أو هبوطاً.
- العروض السريعة (Flash Deals):** التحقق من وجود خصم نشط على الغرفة وطرده من السعر.
- مكافآت الولاء (Loyalty Rewards):** إذا اختار المستخدم تطبيق مكافأة يمتلكها (من `UserRewardInstance`)، يتم خصم مبلغ أو نسبة المكافأة المحددة.
- الضريبة الإجمالية:** تضاف نسبة مئوية ثابتة (15%) على المبلغ الصافي للحصول على `total_price` النهائي المعتمد في جدول الحجزات.

ب. نظام الولاء (Loyalty System)

بعد إتمام العميل للحجز بنجاح، يقوم النظام بترقية نقاطه وتحديث مستوى ولائه، مما يتيح له استرداد نقاطه للحصول على قسائم خصم جاهزة للاستخدام التلقائي في الحجز القادم. الكود موجود في `loyaltyController.js`.

ج. نظام الخرائط والمواقع الحقيقية (Maps & Location)

تم ربط إحداثيات الفنادق بخريطة `Leaflet / OpenStreetMap` عبر مكونات الواجهة الأمامية. عندما يحدد مدير الفندق في `ManagerPortal.jsx` موقع الفندق بالنقر على الخريطة، يتم حفظ `latitude` و `longitude` الحقيقية في

قاعدة البيانات. وعندما يستعرض العميل الفندق في `HotelDetail.jsx` ، يتم عكس الإحداثيات جغرافياً (Reverse Geocoding) عبر واجهة `Nominatim` الخارجية لعرض اسم الدولة والمدينة الدقيق متجاهلاً البيانات الوهمية القديمة، وتُعرض الخريطة بالاعتماد على تلك الإحداثيات.

11. الخلاصة

يمثل مشروع **Smart Hotel Booking** بنية برمجية صلبة وفعالة، تستفيد من قوة Node.js في معالجة طلبات API المتزامنة، وقوة React في واجهات المستخدم الديناميكية المستجيبة. يظهر الكود الفعلي للمشروع نضجاً في التعامل مع العلاقات المعقدة لقاعدة البيانات، والتسعير الديناميكي، وإدارة الأذونات المتعددة بصورة آمنة وموثوقة، مما يجعله نموذجاً تطبيقياً جاهزاً للتوسع المستقبلي.

نهاية التوثيق المصدري المعماري المعتمد
تم المسح والتوثيق آلياً من الملفات الفعلية للمشروع.